(http://parrotprediction.com/)

Dive into WordNet with NLTK

Posted on 24th September 2016 (http://parrotprediction.com/dive-into-wordnet-with-nltk/) by Norbert (http://parrotprediction.com/author/norbert/) in Natural Language Processing (http://parrotprediction.com/category/natural-language-processing/)

It's a common fact that analyzing text documents is a tough nut to crack for computers. Simple tasks like distinguishing whether a sentence has a positive meaning, or that two words mean literally the same require a have a lot of samples and train various machine learning models.

This article will show you how you can increase the quality of generated features and gain new insights about your data.

Enter WordNet

WordNet is a semantically oriented dictionary of English, similar to a traditional thesaurus but with richer structure.

NLTK module includes the English WordNet with 155 287 words and 117 659 synonym sets that are logically related to each other.

It's completely free! In this experiments below, we will use Python 3.5 version (which can be easily installed with PIP).

Begin with importing the WordNet module:

```
>>> from nltk.corpus import wordnet as wn
```

The first worth-understanding concept is a *"synset"*:

> Synset - "*synonym set*" - a collection of synonymous words

We can check what is the synset of the word **motorcar**:

```
>>> wn.synsets('motorcar')
[Synset('car.n.01')]
```

The output means that word **motorcar** has just one possible context. It is identified by `car.n.01` (we will call it "lemma code name") - the first noun (letter `n`) sense of car.

You can dig in and see what are other words within this particular synset:

```
>>> wn.synset('car.n.01').lemma_names()
['car', 'auto', 'automobile', 'machine', 'motorcar']
```

All of the 5 words have the same context - **a car** (more precisely - `car.n.01`).

But as you might expect a word might be ambiguous, for example, **a printer**:

```
>>> wn.synsets('printer')
[Synset('printer.n.01'), Synset('printer.n.02'), Synset('printer.n.03')]
```

You see that there are 3 possible contexts. To help understand the meaning of each one we can see it's definition and provided examples (if are available).

```
>>> for synset in wn.synsets('printer'):
...     print("\tLemma: {}".format(synset.name()))
...     print("\tDefinition: {}".format(synset.definition()))
...     print("\tExample: {}".format(synset.examples()))

    Lemma: printer.n.01
    Definition: someone whose occupation is printing
    Example: []

    Lemma: printer.n.02
    Definition: (computer science) an output device that prints the results of data processing
    Example: []

    Lemma: printer.n.03
    Definition: a machine that prints
    Example: []
```

Like in previous examples, we can see what words (lemmas) are included in each lemma. Here you can see the reason how code names help to avoid the ambiguity. Note that we can call both `lemma_names()` and `lemmas()` on the synset.

```
>>> wn.synset('printer.n.01').lemmas()
[Lemma('printer.n.01.printer'), Lemma('printer.n.01.pressman')]

>>> wn.synset('printer.n.02').lemmas()
[Lemma('printer.n.02.printer')]

>>> wn.synset('printer.n.03').lemmas()
[Lemma('printer.n.03.printer'), Lemma('printer.n.03.printing_machine')]
```

Lexical relations

As you can see a WordNet creates a sort of hierarchy. There are very general and abstract concepts (like *event*, *entity*, *thing*) and a very specific like a *starship*.

NLTK makes it easy to navigate between concepts in different directions by using some special terms.

> Hyponym - a more specific concept

For example let's see what are the hyponyms of lemma `printer.n.03` which is defined as *"a machine that prints"*:

```
>>> machine_that_prints = wn.synset('printer.n.03')
>>> sorted([lemma.name() for synset in machine_that_prints.hyponyms() for lemma in synset.lemmas()])

['Addressograph', 'addressing_machine', 'character-at-a-time_printer', 'character_printer', 'electrostatic_printer', 'impact_printer',
```

We can also go in the opposite way - towards the most general concept.

> Hypernym - a more general concept.

Let's inspect it's value for the example above:

```
>>> [lemma.name() for synset in  machine_that_prints.hypernyms() for lemma in synset.lemmas()]
['machine']
```

In this case, a more abstract term of *printer machine* is just a *machine*.

We can also obtain a top level hypernym (in this case `entity.n.01`) and complete path of words it takes to get to it:

```
>>> machine_that_prints.root_hypernyms()
[Synset('entity.n.01')]

>>> [synset.name() for synset in machine_that_prints.hypernym_paths()[0]]
['entity.n.01', 'physical_entity.n.01', 'object.n.01', 'whole.n.02', 'artifact.n.01', 'instrumentality.n.03', 'device.n.01', 'machine.r
```

Both *hyponyms* and *hypernyms* are called **lexical relations**. They form so-called *"is-a"* relationship between synsets.

There is also another way to navigate through WordNet - from components of items (**meronyms**) or to the things they are contained in (**holonyms**).

> Meronym - denotes a part of something

For meronyms we can take advantage of two NLTK's functions:

- `part_meronyms()` - obtains parts,
- `substance_meronyms()` - obtains substances

Let's see it in action for the word **tree**.

```
>>> tree = wn.synset('tree.n.01')

>>> tree.part_meronyms()
[Synset('burl.n.02'), Synset('crown.n.07'), Synset('limb.n.02'), Synset('stump.n.01'), Synset('trunk.n.01')]

>>> tree.substance_meronyms()
[Synset('heartwood.n.01'), Synset('sapwood.n.01')]
```

On the other side we have **holonyms**:

> Holonym - denotes a membership to something

Like above here we also have 2 functions available - `part_holonyms()` and `substance_holonyms()`.

You can see how they work for words like **atom** and **hydrogen**.

```
>>> wn.synset('atom.n.01').part_holonyms()
[Synset('chemical_element.n.01'), Synset('molecule.n.01')]

>>> wn.synset('hydrogen.n.01').substance_holonyms()
[Synset('water.n.01')]
```

There is also a relationship specific to *verbs* - **entailments**.

> Entailment - denotes how verbs are involved

You can obtain them using the `entailments()` function.

```
>>> wn.synset('eat.v.01').entailments()
[Synset('chew.v.01'), Synset('swallow.v.01')]
```

Similarity

You have seen that words in WordNet are linked to each other in different ways. Given a particular synset you can traverse the whole network to find related objects.

Recall that each synset has one or more parents (*hypernyms*). If two of them are linked to the same root they might have several hypernyms in common - that fact might mean that they are closely related. You can get to it with function `lowest_common_hypernyms()`.

Check what words **truck** and **limousine** have in common:

```
>>> truck = wn.synset('truck.n.01')
>>> limousine = wn.synset('limousine.n.01')

>>> truck.lowest_common_hypernyms(limousine)
[Synset('motor_vehicle.n.01')]
```

You can also examine how *specific* a certain word is, by analyzing it's depth in a hierarchy.

```
>>> wn.synset('entity.n.01').min_depth()
0
>>> wn.synset('car.n.01').min_depth()
10
>>> wn.synset('horse.n.01').min_depth()
14
>>> wn.synset('mare.n.01').min_depth()
15
```

WordNet also introduces a specific metric for quantifying the similarity of two words by measuring shortest path between them. It outputs:

- `range (0,1)` → 0 if not similar at all, 1 if perfectly similar
- `-1` → if there is no common hypernym

Let's try some examples:

```
>>> train = wn.synset('train.n.01')
>>> horse = wn.synset('horse.n.01')
>>> animal = wn.synset('animal.n.01')
>>> atom = wn.synset('atom.n.01')

>>> print("Train => Horse: {}".format(train.path_similarity(horse)))
Train => Horse: 0.058823529411764705

>>> print("Horse => Train: {}".format(horse.path_similarity(train)))
Horse => Train: 0.058823529411764705

>>> print("Horse => Animal: {}".format(horse.path_similarity(animal)))
Horse => Animal: 0.1111111111111111

>>> print("Train => Atom: {}".format(train.path_similarity(atom)))
Train => Atom: 0.09090909090909091

>>> print("Animal => Atom: {}".format(animal.path_similarity(atom)))
Animal => Atom: 0.1111111111111111
```

What's next?

As always I recommend you experiment with NLTK module on your own and try to incorporate some features into your models.

In the first place you can try to:

- use synsets code names instead of words,
- add word's synonyms to as different features,
- calculate how specific each word is (or it's average across a sentence, etc.)

For more information, you can refer to "Natural Language Processing with Python" (http://www.nltk.org/book/) by Steven Bird, Ewan Klein, and Edward Loper.

Tagged Python (http://parrotprediction.com/tag/python/)

(http://parrotprediction.com/author/norbert/)

Norbert

Let's combine software craftsmanship and data engineering skills results to produce some clean and understandable code.

PREVIOUS

So you are eating healthy… Oh, really? (http://parrotprediction.com/so-you-are-eating-healthy-oh-really/)

NEXT

The Tao of Text Normalization (http://parrotprediction.com/the-tao-of-text-normalization/)

Hi, Im Norbert